Weekly Goals

- Finish Weather API scalability and push to main
    - Priority project, I need to have this done by 4/16/2026
    - I will need to talk to my group about implementing this with the general model
- Discuss other methods of scaling with my teammates
    - I will work on the model is nobody else is working on it
    - I will continue to search for better scaling methods if that needs to be found
    - If I need to do general research, I can engage in that
- Should there be no other current features that need to be scaled, work on data creation.
    - Finish weather pipeline and get the next round of training data generated

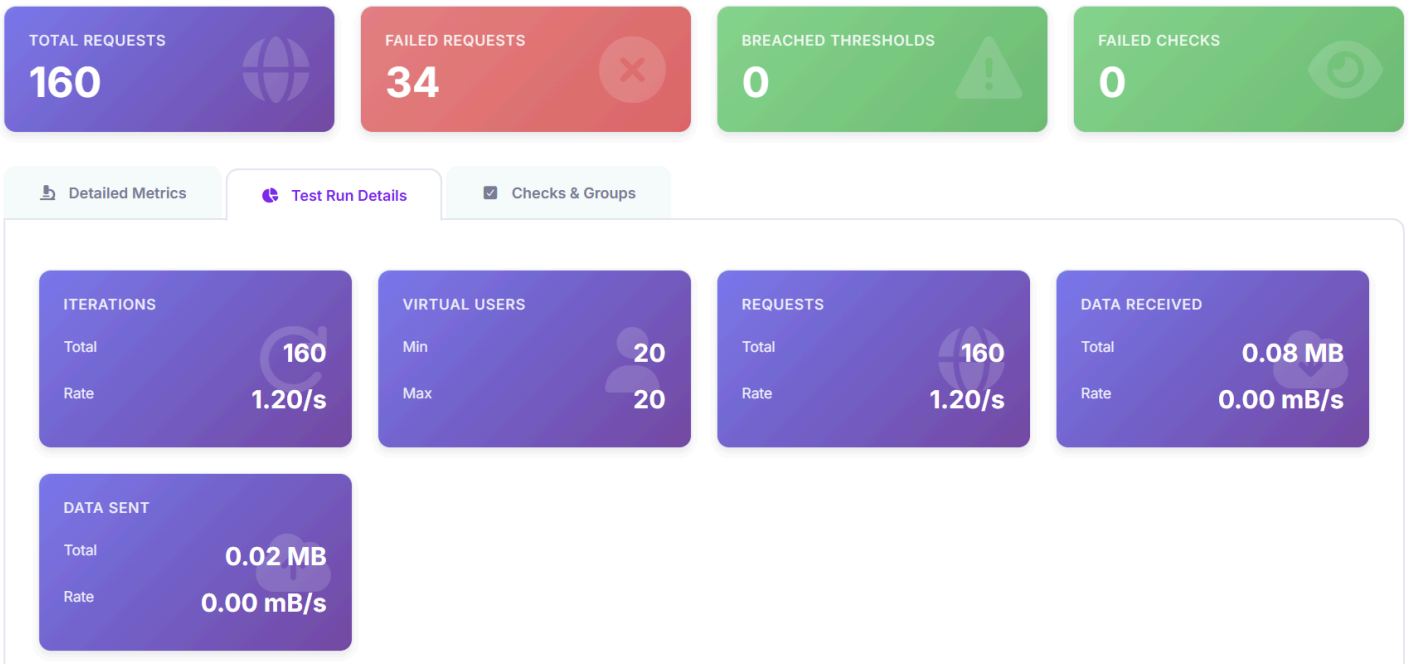
I will work on whatever requires my attention and forward the project. However, I also know that part of the capstone is learning new skills. While I am not necessarily learning new programming scales, the process of scaling is not something I have done before and if other people either feel more equipped to scale a specific part of the project or wish to have the opportunity to try scaling a specific part of the project, I will allow them to do so. If nobody feels this way, I am always willing to do that part myself.

Weather API Scaling

Last week, caching was enabled. This meant that every time a repeated value was asked for (in a ten minute frame) an instantaneous answer could be provided. This, as stated in my last report, scales the weather API in that regard to how fast the server can process requests. The only weakness there is that if we ever have 2 distinct servers, the caches cannot interact but that can be solved if and when a second server is added.

Not the slowdown comes from how many unique roads can be accessed in a limited amount of time. The math is in the last report, but currently the goal is to fetch around 200 unique roads as fast as possible with no errors. The current benchmark test is through k6 with 20 VUs querying every 10 seconds. 60/10 = 6 => 6 * 20 = 120 => 120 * 2 = 240 requests total.

The last test where all roads were ‘unique’ (noncached) had these results:



Only 160 of the 240 requests were made and 34 failed. The goal is to have all requests go through and none fail. If this can be accomplished, I will consider this a general success. From there, I will want to see how far I can push the API until it begins to fail to understand how robust the system has become.

Batch Requests

The idea here is that if individual requests take a long time, then service many requests at once with one connection. This under the assumption that the part of these requests that take the longest is the connection to Open-Meteo and not that actually generating/requesting the data on their end takes the longest (if so I have no control over the time to get the data).

I would need to know if Open-Meteo accepts batch requests, how long these batch requests can be, then work on ironing out the details on actually implementing the batch requesting.

According to one of their own reporting sites, they do allow for batch requests:
https://openmeteo.substack.com/p/weather-data-for-multiple-locations

More specifically, they would need to be organized like this:

```
https://api.open-meteo.com/v1/forecast?latitude=52.52,40.7143&longitude=13.41,-74.006&hourly=temperature_2m
```

So long as they are comma separated like so: <lat1>,<lat2>&<long1>,<long2>

First, I need to ensure that it works. At this stage, a second value is hardcoded into the system:


```
▼ data:
  ▼ 0:
      latitude:                40.710335
      longitude:               -73.99308
      generationtime_ms:       0.1341104507446289
      utc_offset_seconds:      -14400
      timezone:                "America/New_York"
      timezone_abbreviation:   "GMT-4"
      elevation:               27.0 JS: 27
    ▼ current_units:
        time:                  "iso8601"
        interval:              "seconds"
        temperature_2m:        "°C"
        precipitation:         "mm"
        weather_code:          "wmo code"
    ▼ current:
        time:                  "2026-04-16T14:30"
        interval:              900
        temperature_2m:        32.0 JS: 32
        precipitation:         0.0 JS: 0
        weather_code:          3
        weather:               "2 - CLOUDY"
        road_condition:        "1 - DRY"
  ▼ 1:
      latitude:                34.060257
      longitude:               -118.23433
      generationtime_ms:       0.1195669174194336
      utc_offset_seconds:      -25200
      timezone:                "America/Los_Angeles"
      timezone_abbreviation:   "GMT-7"
      elevation:               87.0 JS: 87
      location_id:             1
    ▼ current_units:
```


While not all the data could fit on the screen at this time, two values were returned with weather data. This shows that bulk requesting is possible. I did notice a slowdown upon the request, which I hope will not be indicative of more problems later, but for now I do have a good justification to continue.

The issue with hardcoding is that this is not how the end project is going to interact with the project. Instead, multiple users are going to need to request individual latitude and longitude coordinates and then make one bulk request. Thus, we need to implement a system that gets the individual requests that users make, hold these latitude longitude coordinates for x time or until y requests are made and then fetches all the sites. From there, once everything is returned, cache everything.

We do need to keep in mind when duplicate requests are in the cache, trying to make them wait for a cache as opposed to having one bulk request full of duplicates. Another thing to keep in mind is that if we are running off making a request every x time (I have read that the recommended time is in ml so I will do 300 for now), we need to have the request recheck the queue before running to make sure the last request did not send it.

I do know that this will not be as fast for individual requests given that we are waiting 300 ms (or some other small arbitrary amount) but the end goal is increased speed for more users, so this should not be an issue.

Once done, I need to check 3 things:
- Single requests still work
- There is a speedup/more reliability
- Caching still works.

While I have been using the Single Source of Truth< i want to ensure that Cursor has not broken any of the features in an attempt to implement new ones. First things first, a single normal request will be sent. We will ensure that it still runs, then repeat it to check for individual caching.


```
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.09238719940185547
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               27.0 JS: 27
current_units:
  time:                  "iso8601"
  interval:              "seconds"
  temperature_2m:        "°C"
  precipitation:         "mm"
  weather_code:          "wmo code"
current:
  time:                  "2026-04-16T15:00"
  interval:              900
  temperature_2m:        31.7
  precipitation:         0.0 JS: 0
  weather_code:          3
  weather:               "2 - CLOUDY"
  road_condition:        "1 - DRY"
cached:                  false
```


This runs, but it is not cached which is good for testing. Running again yields:

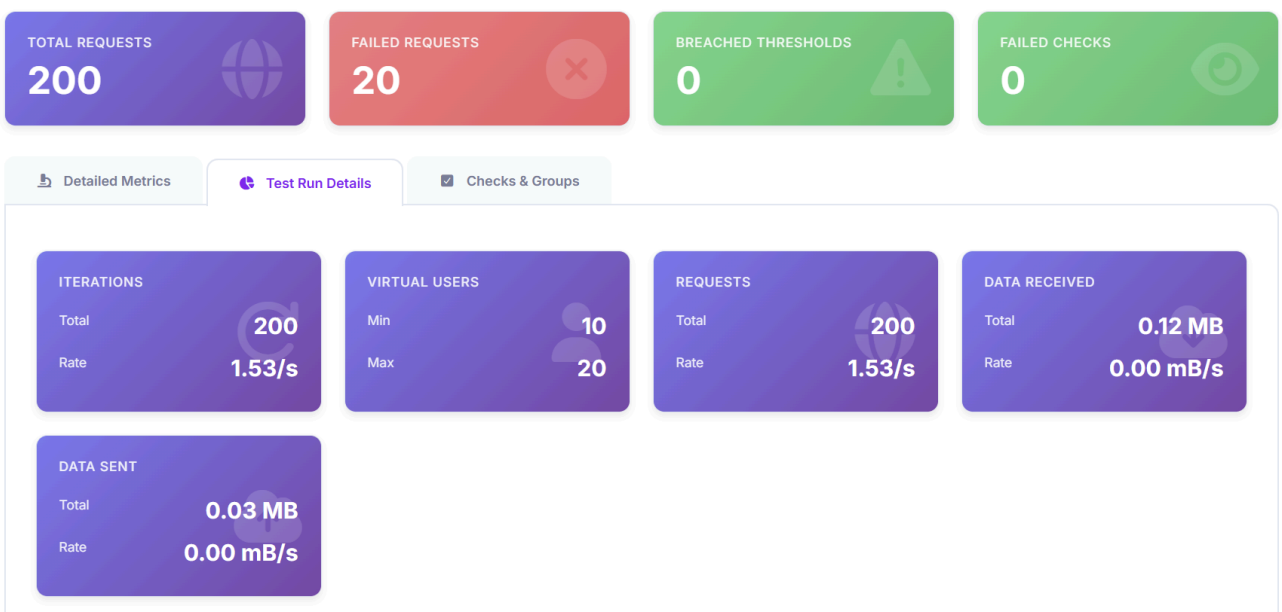
```
Save  Copy  Collapse All  Expand All  Filter JSON
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.09238719940185547
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               27.0 JS: 27
current_units:
  time:                  "iso8601"
  interval:              "seconds"
  temperature_2m:        "°C"
  precipitation:         "mm"
  weather_code:          "wmo code"
current:
  time:                  "2026-04-16T15:00"
  interval:              900
  temperature_2m:        31.7
  precipitation:         0.0 JS: 0
  weather_code:          3
  weather:               "2 - CLOUDY"
  road_condition:        "1 - DRY"
cached:                  true
```


Which was cached with a noticeable speedup. Thus, we can confirm that individual requests go through just fine and caching works just fine for single values at least. I am going to run under the assumption that caching will not be an issue for multiple requests, but testing will reveal that.

Testing

Now that this has been finished, the next thing to do is to run through and do a k6 test on the new weather api function. The details of the test are above, toward the weather API scaling header. However, this test that will be run will have some issues.

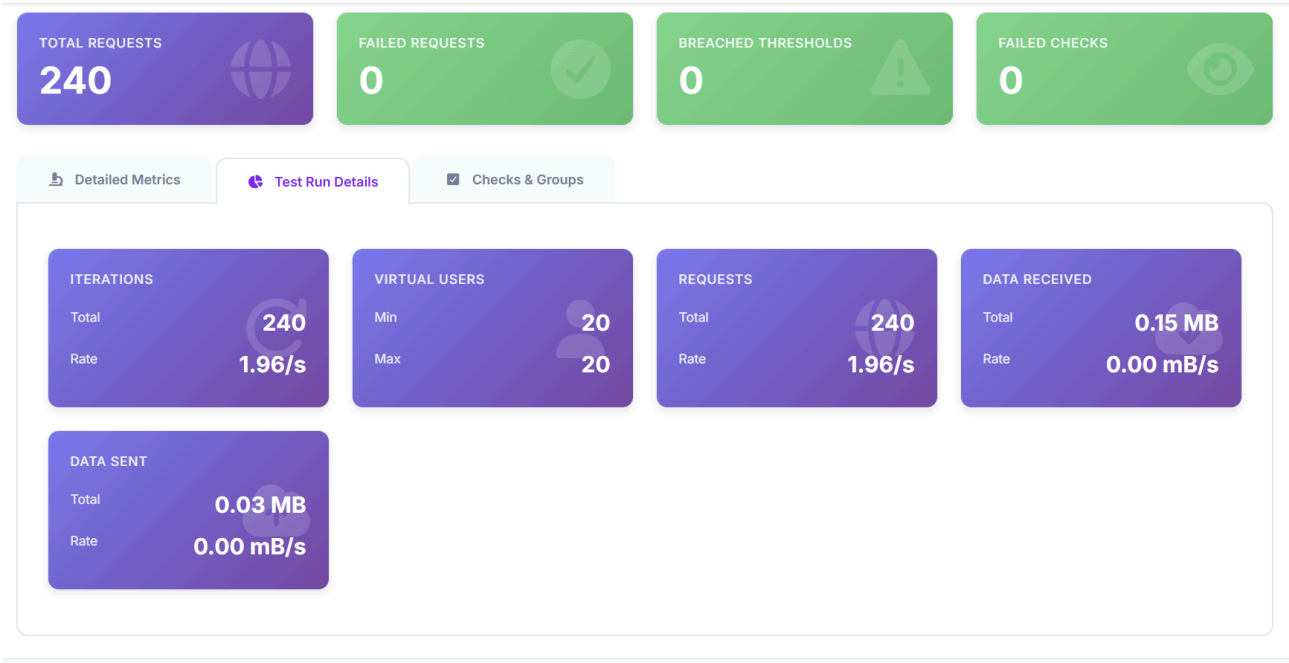
Given that caching exists, any repeat values will be cached. This means that some 200 different latitude/longitude coordinates will need to be written out (or randomly generated) but all 200 will likely not execute at the same time. The worst case, which is the ideal to test



This is the result of the first test. These are not amazing results in my eyes. While this is around 54 more successful responses (34 + 20 from 160 total to 180 completed), this does not hit the goal of at least 240 responses with 0 failures that I was looking for. Watching the test go showed that both requests were finished almost simultaneously around 11-15 seconds after the requests were made. Therefore, I am of the mind to believe that it is the free tier of Open-Meteo that is slowing down service and not necessarily the code.

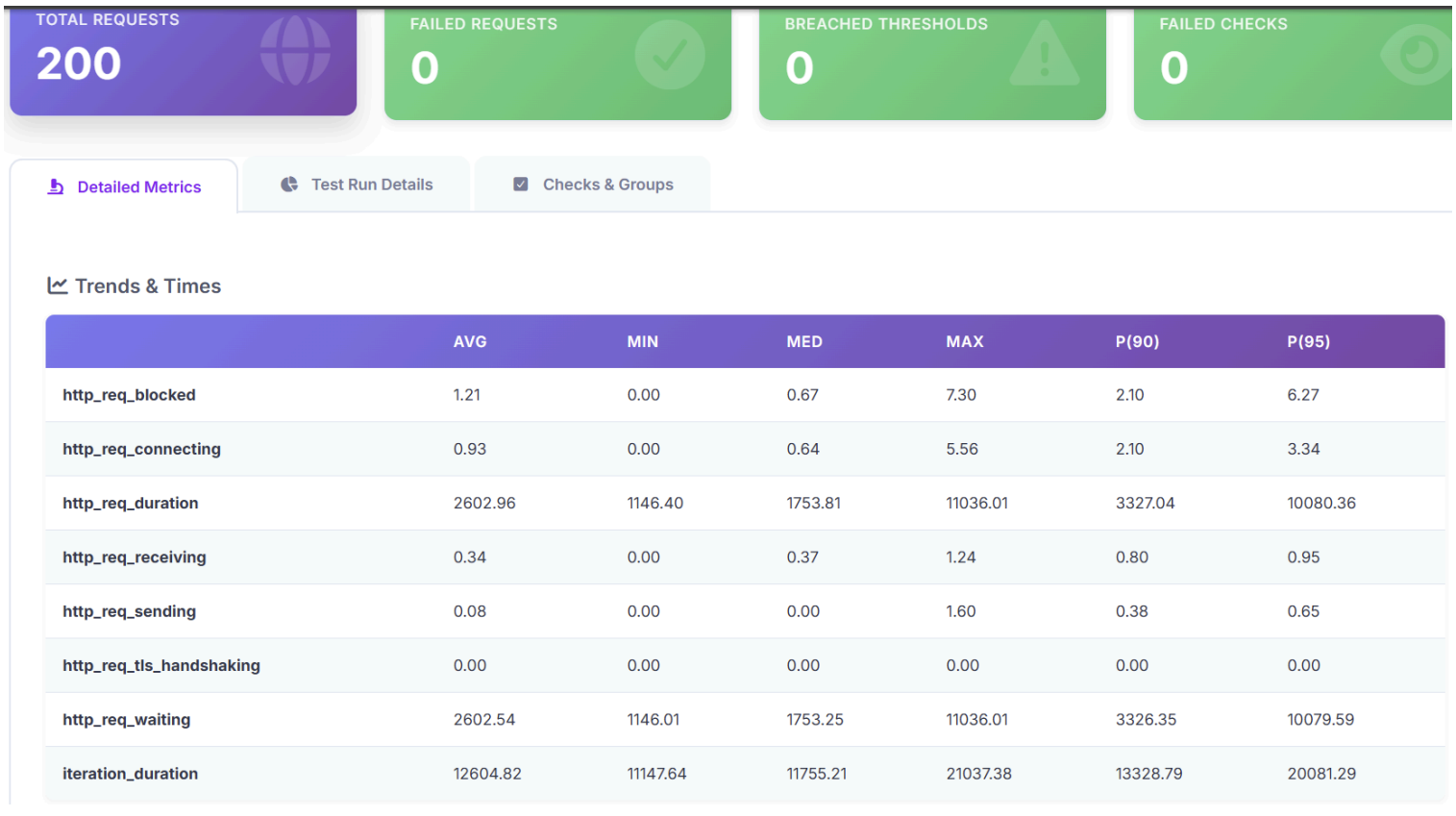
I want to run another test with a small batch size, 5 instead of 10, and then see what the results are. I hope that will mean better results as Open-Meteo will have a smaller response time. However, we will see.

While waiting for the cache to expire, I figured I would ensure that the cached data was still working despite my tampering with the bulk requests and the results are below.

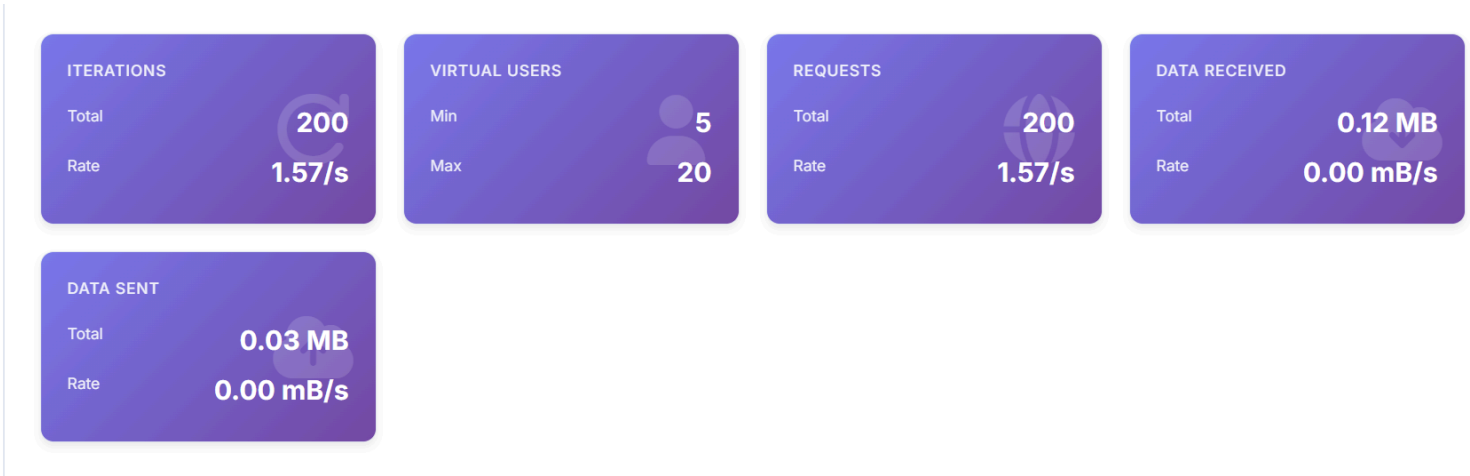


The difference is test and response time does speak for itself, caching does work. I do notice that unlike the last test, we did not hit 2 requests in a second which means that there is some overhead somewhere in the code prior to caching maybe. This is not catastrophic, but it may be worth investigating later.

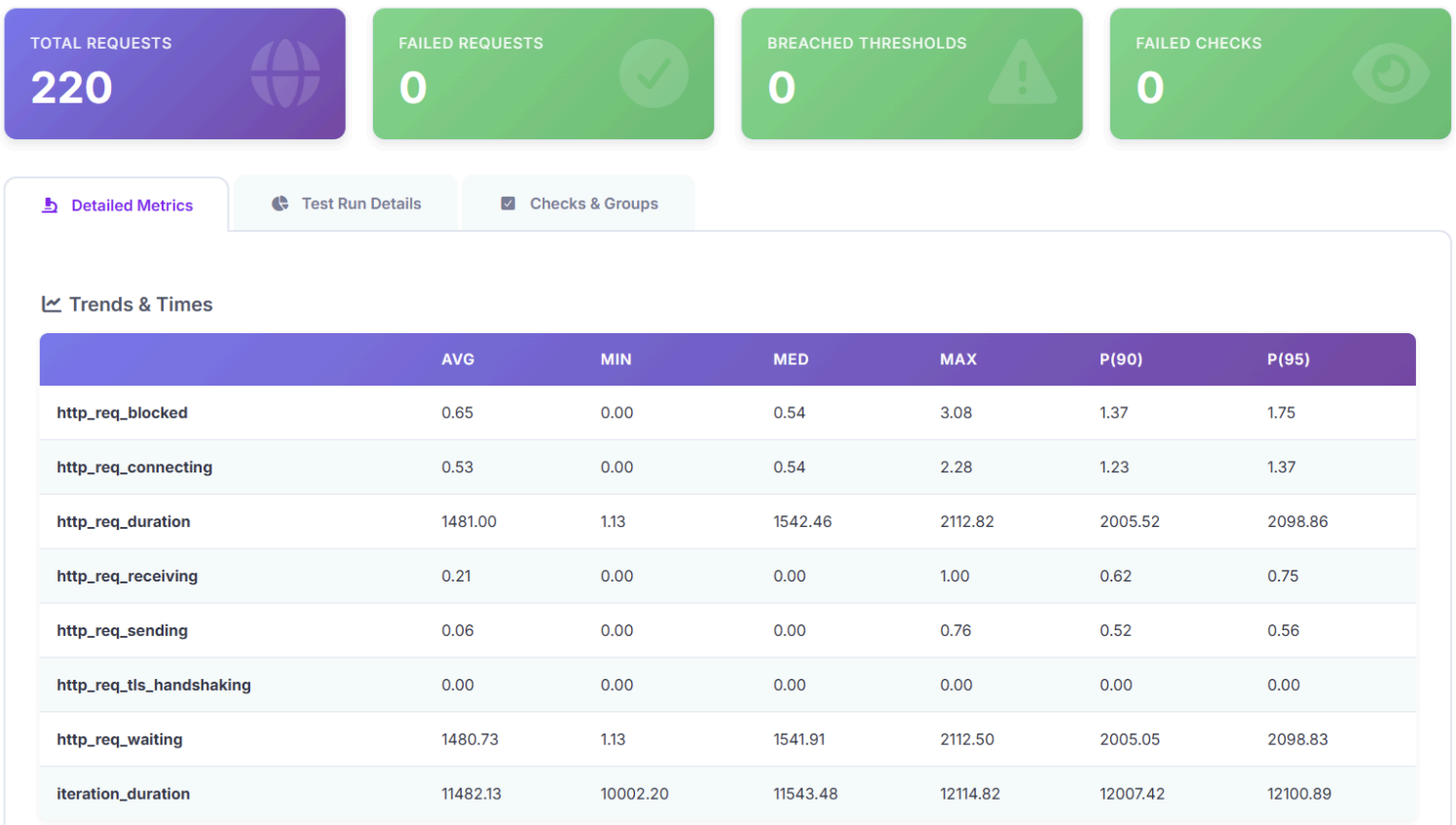
The batch of 5 results came in like this:



	AVG	MIN	MED	MAX	P(90)	P(95)
http_req_blocked	1.21	0.00	0.67	7.30	2.10	6.27
http_req_connecting	0.93	0.00	0.64	5.56	2.10	3.34
http_req_duration	2602.96	1146.40	1753.81	11036.01	3327.04	10080.36
http_req_receiving	0.34	0.00	0.37	1.24	0.80	0.95
http_req_sending	0.08	0.00	0.00	1.60	0.38	0.65
http_req_tls_handshaking	0.00	0.00	0.00	0.00	0.00	0.00
http_req_waiting	2602.54	1146.01	1753.25	11036.01	3326.35	10079.59
iteration_duration	12604.82	11147.64	11755.21	21037.38	13328.79	20081.29



This is not too different from when batches of 10 were being sent over, just now the last 4 requests to Open-Meteo (20 individual requests) went through. Once again I was watching all the tests run and I saw large insidious jumps in completed requests in batches of 20. I believe that the current issue is that we are using a non-commercial and non-priority server and thus this is the best performance possible from Open-Meteo. I want to run one final test where the wait time is 2 seconds as opposed to 0.3 seconds. I have a feeling that that may change quite a bit. It is too long to use in the project, but it will be good enough for just a demo.



	AVG	MIN	MED	MAX	P(90)	P(95)
http_req_blocked	0.65	0.00	0.54	3.08	1.37	1.75
http_req_connecting	0.53	0.00	0.54	2.28	1.23	1.37
http_req_duration	1481.00	1.13	1542.46	2112.82	2005.52	2098.86
http_req_receiving	0.21	0.00	0.00	1.00	0.62	0.75
http_req_sending	0.06	0.00	0.00	0.76	0.52	0.56
http_req_tls_handshaking	0.00	0.00	0.00	0.00	0.00	0.00
http_req_waiting	1480.73	1.13	1541.91	2112.50	2005.05	2098.83
iteration_duration	11482.13	10002.20	11543.48	12114.82	12007.42	12100.89

This is even better. I do think this proves that the issue is upstream with Open-Meteo access times. The completed test number came back in batches this time, but still pretty quick. I believe that this has become an issue with ‘how fast can Open-Meteo process our request.’

I would like to talk with the team and the professor both at the meeting on the 16th and 17th to see if they are in agreement. There is a world in which someone has a solution. However, I have hit the limit of what I understand from generated code and thus am going to push this branch and move on. Before fully moving on I would like to run a more average case test than a worst case all the time test. This will be each VU randomly generating coordinates and then sending them forth. This allows for duplicates and for caching to kick in and make a difference.After running, it made no real difference in processing time.

Data Generation

I have completed the weather API before the meeting on 4/16/2026. As usual, while I trust the team is working on something, I am not yet sure where everyone is on their section of the project. It is possible that I could implement my new weather API into the model's path as well as beginning to test it, but last I was aware, that model was still in development. Once the meeting on 4/16/2026 happens, then I will know if I have a path to follow in regards to increasing scalability or if I will go back to generation.

In the meantime, I would like to continue to work on the Training Data. At the very least before phase 5 ends, I want to be able to generate a second batch of training data based off of weather to work with. When I had last worked on it I had managed to download the csvs of historic weather for specific cities/counties. However, as was explored in the week 6 report, the csvs downloaded do not describe all of the cities/counties in Texas. Unlike Open-Meteo, latitude/longitude does not always have a corresponding station.

As an aside, should I have more time on this project I would love to simplify this by using Open-Meteo. I consider it to be a 'good enough' weather option that even though NOAA is directly reporting, its accuracy is less important than coverage. However, to automate it would be difficult and time consuming both to generate the data. 200 points for the Dallas area alone takes around 3 minutes, being more specific would be awful and Texas even worse. It would also take me at least a week to overhaul the existing system I have in place. While I am aware that is a bit of a sunk cost fallacy, I also only have 2-3 more weeks to finish this project and I already don't think I can get everything done.

Regardless, to fix the problem, I want to map the cities/counties that do not exist to their closest stations. This does not make weather perfect, but I consider it to be a good enough approach.

Location Mapping

The immediate problem is that we know what cities are represented, but we do now know what counties are not represented. Hard coding is an option, but not long term for the sake of automation.

I think the ideal move would be to preprocess the positive samples and then collect all the names of the cities/counties. These incidents initially come with latitude and longitude, but I don't want the coordinates of the crash as we don't want to map the crash to the nearest city/county (thought that would be better but it is an insane amount of overhead), we want to map the general county/city the crash occurred in to the nearest station.

First, a csv called stations.csv should be made, in it all the stations that exist will be placed as well as their coordinates, and a column called closest station which will just be the name of the city or county that represents their closest station.

To do this, we need to ensure that latitude and longitude are not removed in the geocoding step when we remove unnecessary functions. Then they need to be saved and added to a new file after successful download during the download step.

When done it will look like this

```
name,lat,lon,closest_station,station_flags
Mesquite,32.747,-96.531,Mesquite,1
Lancaster,32.579,-96.719,Lancaster,1
Dallas,32.838,-96.836,Dallas,1
Irving,32.897,-97.022,Irving,1
Addison,32.969,-96.836,Addison,1
```

Station_flsgs will be what we use to compare cities or counties without a station to. After preprocessing of roads, a list of total cities or counties can be collected and then paired with this list. Any that exist will be skipped (so if Mesquite is checked again it will just be jumped over)

From here, This is as much as can be done until road processing is done, we at least need a list of the crash reports so we can get a list of all the cities that need to be represented. I do not intend to wait until the road processing has

been completed to continue on with this project. I do have a single year worth of data. It exists in both preprocessed form and as training data. I will continue on the normal flow. For now, we assume that the large amount of csvs that NOAA provided are preprocessed and we move on.

The next thing we do is take the preprocessed data and create an array of cities/counties (depending on the cities variable in the CreateTrainingData.py file. Once done, we get the list of cities/counties with a station. We then loop through the list of counties/cities and geocode to get their latitude/longitude coordinates. We then compare with all the rows and insert them into the file along with the name of the station that is the closest.

The NOAA data does include 114 lines of data that are ‘outside of city limits.’ For now I am going to map these to Dallas (regardless of city vs county) but a better solution can be made later. Given that this is only 114 lines out of almost 49 thousand, I am not too worried one way or the other.

The issue here is that the census data that is used to geolocate takes latitude and longitude to spit out city and county names and there is not a good option to do the exact reverse of that. Most GPS or geolocation services can get city locations, but not county locations. Given that so far we are only working on the scale of cities, this is good enough, but I do want to come back and fix it later. I may have to save city/state/country data when running this data to get county information. At least for now, I can assume that the pipeline will always produce Texas information so that can be hard coded in.

I think the best thing to do would be to save city name and county name inside of stations.csv so that the fields would look something like this. That way I can pick an arbitrary city from the crash data to use as its center. This is not an ideal solution, but given that looking up county latitude and longitude coordinates is not a simple task, this may be a good enough solution

```
city,name,lat,lon,closest_station,station_flags
```

For now, I am going to use Open-Meteo and it’s geocoding api abilities. I have come to enjoy it and while there is no way I have time to do so, I would love to have the entire project run on it, but it’s already built out with weather so I have no choice.

The main issue that this has is that a state cannot be fed into the API so that it will pull cities inside a state. That does not work too bad for Dallas, but does leave out 2 of the cities that lay within that area and would be harder for counties to be grabbed if the valid city does not show up in the first ten results.

I have found another geolocation service called Nomination that may work better for this particular use and does take in State as a parameter. This does seem to hit the locations that open-meteo could not, but it does seem prone to being rate limited. Given how strict they are with rate limiting, this may become an issue down the line.

However, if it continues to provide good information, comparing it to latitude and longitude that already exists should be simple.

4/16/2026 Meeting

In a very short meeting, we discussed some level of progress. For now the model still seems to be in some sort of final development and iteration. The weather api will be implemented by Ben, the man who has been making the model so I will not worry about that.

I did bring up a few ideas about scaling. I have mentioned my fears about Open-Meteo being stuck by the free tier we are using. No other ideas were brought forward, I still do intend to bring the issue up on the 4/17/2026 meeting to get the opinion of our professors.

I also brought up scaling the model. It will be a bit difficult, but my teammates in charge of the model seem to have some ideas about math regarding both if and how to store cached data as well as reacting to user reported accidents. Unless told otherwise, I am going to continue to work on data generation for now. If new ideas come up tomorrow, I will discuss with the group on how to pursue them.

9 Dennis Keithly

I have been asked to create new training data that both combines street name and section as well as making them separate. I do believe that training a road on sections independent of name means that the model will come to false conclusions about sections based on number, but I do understand the thought. Right now sections are being modeled as ‘downtown area going south’ which would hold unique weight, but the numbers do not actually have any corresponding weight or value. Any similarities would likely be circumstantial or coincidental. However, I do not understand the model and I trust he has a plan so I will do my best to provide tailored data that fits his vision.

Next Week’s Goals

- Finish improved weather generation
    - Finish mapping
    - Finish preprocessing
    - Generate data set with and without road section integrated
- Look into AI model scaling
- Update SSoT
- Begin looking into presentation preparation